CS3081 – Cloud Computing Mini Project

Name : Sujeet kumar
Roll : 2301032014

MetaData And what I learnt :-

(1.) Knowledge Distillation :

→ Learnt that large models (Phi-3.5 3.8B) can transfer domain-specific knowledge to ultra-small models (Pythia-70M). The student model does not need to learn general conversation, only the 4-key JSON command parsing schema. On CPU, it's achieved 98% JSON validity and 75% full-frame accuracy.

(2.) Decoupling Storage, and Compute :

→ Discovered why large model should never be baked inside Docker storing weights in S3 (SeaweedFS) and dynamically fetching them at pod startup reduced our Docker image to 1.18 GB and decoupled code CI/CD from model versioning.

(3.) Kubernetes Health Probes :

→ Understood the critical difference b/w (/healthz) and readiness (/readyz) probes. Readiness probes prevent traffic from hitting a pod until model weights are loaded into memory, ensuring zero dropped requests during zero dropped requests during rolling updates.

(4.) Stateless Network Security (VLSM & NACL) :

→ Mastered binary subnetting for the 10.20.0.0/16 block (/22, /24, /25, /27).
Learned that stateless firewalls inspect packets individually, requiring explicit rules for ephemeral return ports (1024-65535).

Challenges faced & Technical Solutions :

1. Challenge-1 : Docker Image Bloat (>9.5 GB)

Solution : Pinned CPU-only wheels using `--index-url : https://download.pytorch.org/whl/cpu`, shrinking the image to 1.18 GB.

2. Challenge-2 : Traffic drops (502 Bad Gateway)

Solution : Configured /readyz to return HTTP 503 until the model is loaded in memory, achieving 0.00% dropped requests during rollout.

3 Challenge-3 : Stateless NACL Dropping Return Packets

Solution : Added explicit allow rules for ephemeral return traffic (ports 1024-65535) and placed the abusive block (198.51.100.0/24) at Rule 100 before the web allow rules.

4 Challenge-4 : Teacher Model Data Noise :

Solution : Created an automated cleaning pipeline (worker/cleanify) enforcing rules (a)-(e) to produce a verified dataset with SHA-256 integrity.

Findings, Observations, & Future Improvements :—

1. Finding 1 : Lightweight footprint

Python - FOM used only 42.12 MB of peak RAM (less than 42% of the 1GB limit), and handled requests in ~29 ms median latency under 4 concurrent users.

2. Finding 2 : Unit Economics

Using `rate_cards.json`, serving 1,000 requests costs only 0.0620 (6.2 paise) with 2 active replicas.

3. Improvement 1 : Multi-worker Interference.

Currently, Uvicorn runs a single process per pod. In production, running multiple workers or ONNX Runtime would increase throughput beyond 35 req/sec.

4 Improvement 2 : Scale-to-Zero with KEDA

Use KEDA to scale-down to 0 replicas during idle periods to eliminate idle compute costs.

5. Improvement 3 : LoRA Fine-Tuning

A slightly larger 0.5B model fine-tuned with LoRA would resolve complex compound slots while staying within a 500MB footprint.